📘 Día 20

Python PIP - Administrador de paquetes de Python

1.¿Qué es PIP?

PIP significa programa de instalación preferido. Usamos *pip* para instalar diferentes paquetes de Python. El paquete es un módulo de Python que puede contener uno o más módulos u otros paquetes. Un módulo o módulos que podemos instalar en nuestra aplicación es un paquete. En programación, no tenemos que escribir todos los programas de utilidad, sino que instalamos paquetes e importamos a nuestras aplicaciones.

2.Instalación de PIP

Si no ha instalado pip, déjenos instalarlo ahora. Vaya a su terminal o solicitud de comando y copie y pegue esto:

asabeneh@Asabeneh:~$ pip install pip

Compruebe si el pip está instalado escribiendo:

pip --version

asabeneh@Asabeneh:~$ pip --version

pip 21.1.3 from /usr/local/lib/python3.7/site-packages/pip (python 3.9.6)

Como puede ver, estoy usando la versión 21.1.3 de pip, si ve un poco de número un poco más abajo o por encima de eso, significa que tiene pip instalado.

Verifiquemos algunos de los paquetes utilizados en la comunidad Python para diferentes propósitos. Solo para hacerle saber que hay un montón de paquetes disponibles para su uso con diferentes aplicaciones.

3.Instalación de paquetes con pip.

Tratemos de instalar *numpy*, llamado Python numérico. Es uno de los paquetes más populares en el aprendizaje automático y la ciencia de datos.

- NumPy es el paquete fundamental para la computación científica con Python. Contiene, entre otras cosas:
    - Un poderoso objeto de matriz N-dimensional
    - Funciones sofisticadas (de radiodifusión)
    - Herramientas para integrar código C/C++ y Fortran
    - Álgebra lineal útil, transformación de Fourier y capacidades de números aleatorios

asabeneh@Asabeneh:~$ pip install numpy

Pandas es una biblioteca de código abierto con licencia BSD que proporciona estructuras de datos de alto rendimiento y fáciles de usar y herramientas de análisis de datos para el lenguaje de programación Python. Instalemos al hermano mayor de numpy, *pandas* :

```
asabeneh@Asabeneh:~$ pip install pandas
```

Esta sección no se trata de numpy ni pandas, aquí estamos tratando de aprender a instalar paquetes y cómo importarlos. Si es necesario, hablaremos de diferentes paquetes en otras secciones.

Importemos un módulo de navegador web, que puede ayudarnos a abrir cualquier sitio web. No necesitamos instalar este módulo, ya está instalado por defecto con Python 3. Por ejemplo, si desea abrir cualquier número de sitios web en cualquier momento o si desea programar algo, se puede utilizar este módulo *webbrowser*.

```
import webbrowser # web browser module to open websites

# list of urls: python
url_lists = [
    'http://www.python.org',
    'https://www.linkedin.com/in/asabeneh/',
    'https://github.com/Asabeneh',
    'https://twitter.com/Asabeneh',
]

# opens the above list of websites in a different tab
for url in url_lists:
    webbrowser.open_new_tab(url)
```

4. Desinstalar paquetes

Si no desea conservar los paquetes instalados, puede eliminarlos utilizando el siguiente comando.

```
pip uninstall packagename
```

5. Lista de paquetes

Para ver los paquetes instalados en nuestra máquina. Podemos usar pip seguido de la lista.

```
pip list
```

6. Paquete de espectáculo

Para mostrar información sobre un paquete

```
pip show packagename
```

7. pip freeze — Explicación detallada

1. ¿Qué es pip freeze?

Definición: Es un comando que lista todos los paquetes instalados en tu entorno Python actual junto con sus versiones exactas.

```
pip freeze
```

2. ¿Para qué sirve?

Imagina que desarrollas un proyecto en tu computadora y funciona perfectamente. Luego quieres:

- Compartirlo con un compañero
- Subirlo a un servidor
- Trabajar en otra computadora

¿Cómo sabe la otra persona qué librerías necesita instalar? Con pip freeze creas una "lista de compras" de todas las librerías necesarias.

3. Ejemplo práctico completo

Paso 1: Crear proyecto y entorno virtual

```
# Crear carpeta del proyecto mkdir
mi_proyecto_web
cd mi_proyecto_web

# Crear entorno virtual python3 -m venv
venv

# Activar entorno virtual source
venv/bin/activate
```

Paso 2: Instalar librerías para el proyecto

```
(venv) $ pip install flask (venv) $ pip install
requests (venv) $ pip install pandas
```

Paso 3: Ver qué librerías se instalaron

```
(venv) $ pip freeze
```

Salida:

```
blinker==1.7.0 certifi==2023.11.17
charset-normalizer==3.3.2
click==8.1.7
```

```
Flask==3.0.0 idna==3.6
itsdangerous==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.3
numpy==1.26.2
pandas==2.1.4
python-dateutil==2.8.2
pytz==2023.3.post1
requests==2.31.0 six==1.16.0
tzdata==2023.4 urllib3==2.1.0
Werkzeug==3.0.1
```

> *¿Por qué hay tantas? Instalaste 3 paquetes: flask, requests, pandas. Pero cada uno tiene dependencias (otros paquetes que necesitan). pip freeze muestra TODOS los paquetes instalados.*

Paso 4: Guardar en archivo requirements.txt

```
(venv) $ pip freeze > requirements.txt
```

¿Qué hace este comando?

- pip freeze: Lista los paquetes
- >: Operador de redirección (guarda la salida en un archivo)
- requirements.txt: Nombre del archivo donde se guarda Ahora

tu proyecto tiene esta estructura:

```
mi_proyecto_web/
|-- venv/
|-- app.py
|-- requirements.txt                <- Archivo creado
```

4. ¿Cómo se usa el archivo requirements.txt?

Escenario 1: Tu compañero quiere ejecutar tu proyecto

```
# 1. Clonar/descargar tu proyecto cd
mi_proyecto_web

# 2. Crear su propio entorno virtual python3 -m venv
venv

# 3. Activar el entorno virtual source
venv/bin/activate

# 4. Instalar TODAS las librerias con un solo comando (venv) $ pip install -r
requirements.txt
```

¿Qué hace pip install -r requirements.txt?

- Lee el archivo línea por línea
- Instala cada paquete con la versión exacta especificada
- Tu compañero tendrá exactamente las mismas librerías que tú

Escenario 2: Desplegar en un servidor

```
# En el servidor
git clone https://github.com/tuusuario/mi_proyecto_web.git cd mi_proyecto_web
python3 -m venv venv source
venv/bin/activate
pip install -r requirements.txt                    <- Instala todo automaticamente python app.py
                    <- El proyecto funciona igual que en tu computadora
```

5. Análisis de la salida del ejemplo

```
asabeneh@Asabeneh:~$ pip freeze docutils==0.11
Jinja2==2.7.2
MarkupSafe==0.19
Pygments==1.6 Sphinx==1.2.2
```

Paquete	Versión	Significado
docutils	0.11	Librería para procesar documentación
Jinja2	2.7.2	Motor de plantillas (usado por Flask, Sphinx)
MarkupSafe	0.19	Dependencia de Jinja2 (escape de HTML)
Pygments	1.6	Librería para resaltar código
Sphinx	1.2.2	Generador de documentación

Formato: nombre_paquete==versión_exacta

El símbolo == significa "versión exacta": Flask==3.0.0 instala *exactamente* la versión 3.0.0, no la 3.0.1 ni la 2.9.9.

6. Flujo completo: De desarrollo a producción

```
+                                                    +
|   TU  COMPUTADORA (Desarrollo)                     |
|                                                    |
|    1.  Crear entorno virtual                       |
|        python3 -m venv venv                        |
|                                                    |
|    2.  Instalar librerias                          |
|        pip install flask pandas                    |
|                                                    |
|    3.  Desarrollar el proyecto                     |
|        app.py, main.py, etc.                       |
|                                                    |
|    4.  Congelar dependencias                       |
|        pip freeze > requirements.txt               |
|                                                    |
|    5.  Subir a GitHub/GitLab                       |
|        git push origin main                        |
|                                                    |
+                                                    +
                          |
```

```
                                | Internet v
+                                                                  +
|    SERVIDOR (Produccion)                                         |
|                                                                  |
|    1. Descargar proyecto                                         |
|          git clone ...                                           |
|                                                                  |
|    2. Crear entorno virtual                                      |
|          python3 -m venv venv                                    |
|                                                                  |
|    3. Instalar TODAS las librerias                               |
|          pip install -r requirements.txt                         |
|          ^                                                       |
|          |-- Lee el archivo y instala TODO                       |
|                                                                  |
|    4. Ejecutar aplicacion                                        |
|          python app.py                                           |
|                                                                  |
+                                                                  +
```

7. Comandos importantes

Crear requirements.txt
pip freeze > requirements.txt

Instalar desde requirements.txt pip install -r
requirements.txt

Ver paquetes instalados (sin guardar) pip freeze

Ver paquetes con detalles pip list

Diferencia entre pip freeze y pip list:

# pip freeze (formato para requirements.txt)
Flask==3.0.0 requests==2.31.0

pip list (formato legible para humanos) Package
Version
- - - - - - - - - - - - - - - - - -
Flask 3.0.0
requests 2.31.0

8. Buenas prácticas

Lo que DEBES hacer:

1. Siempre crear entorno virtual para cada proyecto python3 -m venv venv

2. Activar el entorno antes de instalar source venv/bin/activate

3. Instalar solo lo necesario

pip install flask requests

4. Generar requirements.txt pip freeze >
requirements.txt

5. Incluir requirements.txt en tu repositorio Git git add requirements.txt
git commit -m "Add dependencies"

Lo que NO debes hacer:

NO hacer pip freeze sin entorno virtual activado # (incluira TODOS los paquetes del sistema)

NO editar requirements.txt manualmente sin saber # (pip freeze lo hace automaticamente)

NO compartir la carpeta venv/
(es pesada e innecesaria, solo comparte requirements.txt)

9. Ejemplo real: Proyecto Flask

Tu proyecto:

```
mi_app_flask/
|-- venv/                    <- NO se comparte
|-- app.py
|-- templates/
|     |-- index.html
|-- requirements.txt         <- SI se comparte
|-- README.md
```

Contenido de requirements.txt:

Flask==3.0.0 Werkzeug==3.0.1
Jinja2==3.1.2 click==8.1.7
itsdangerous==2.1.2
MarkupSafe==2.1.3

Tu compañero hace:

1. Clonar repositorio
git clone https://github.com/tu/mi_app_flask.git cd mi_app_flask

2. Crear entorno virtual python3 -m
venv venv source venv/bin/activate

3. Instalar dependencias
pip install -r requirements.txt

4. Ejecutar
python app.py

Funciona! Tiene las mismas librerias que tu

10. Preguntas frecuentes

¿Debo incluir venv/ en Git?

NO. Solo incluye requirements.txt.

- La carpeta venv/ puede pesar 100+ MB
- Cada persona puede recrearla con pip install -r requirements.txt
- Puede causar conflictos entre diferentes sistemas

operativos En .gitignore:

```
venv/
__pycache__/
*.pyc
```

¿Qué pasa si actualizo una librería?

```
# Actualizar Flask a la ultima version pip install --upgrade
Flask

# Regenerar requirements.txt pip freeze >
requirements.txt

# Ahora requirements.txt tiene la nueva version
```

¿Puedo instalar solo algunas librerías de requirements.txt?

No directamente, pero puedes crear un archivo temporal:

```
# Crear requirements_minimal.txt con solo lo basico echo "Flask==3.0.0" >
requirements_minimal.txt
echo "requests==2.31.0" >> requirements_minimal.txt

# Instalar solo esas
pip install -r requirements_minimal.txt
```

11. Resumen ejecutivo

Comando	Acción
pip freeze	Muestra lista de paquetes instalados
pip freeze > requirements.txt	Guarda la lista en un archivo
pip install -r requirements.txt	Instala todos los paquetes del archivo
pip list	Muestra paquetes en formato tabla

12. Analogía final

requirements.txt es como una receta de cocina:

```
Receta: Pastel de Chocolate
- Harina (2 tazas)        <- Flask==3.0.0
- Azucar (1 taza)         <- requests==2.31.0
- Huevos (3)              <- pandas==2.1.4
- Chocolate (200g)        <- numpy==1.26.2
```

---

8. Paquetes y módulos en Python

Un **paquete** puede contener uno o más módulos relevantes.

Un **módulo** puede contener múltiples objetos, tales como clases, funciones, etc.

Un paquete es en realidad una carpeta que contiene uno o más archivos de módulo. Vamos a crear un paquete llamado mypackage:

Estructura de carpetas del paquete

```
- mypackage
    |-- __init__.py
    |-- arithmetic.py
    |-- greet.py
```

9.Lectura desde URL

A estas alturas, ya está familiarizado con cómo leer o escribir en un archivo ubicado en su máquina local. A veces, nos gustaría leer de un sitio web usando url o desde una API. La API significa interfaz de programa de aplicación. Es un medio para intercambiar datos estructurados entre servidores principalmente como datos json. Para abrir una conexión de red, necesitamos un paquete llamado *solicitudes:* permite abrir una conexión de red e implementar operaciones CRUD (crear, leer, actualizar y eliminar). En esta sección, cubriremos solo el mineral de lectura obteniendo parte de un CRUD.

Instalemos *peticiones* :

asabeneh@Asabeneh:~$ pip install requests

Veremos *los* métodos *get*, *status_code*, *headers*, *text* y *json* en el módulo *de solicitudes:*

- *get()*: para abrir una red y obtener datos de url - devuelve un objeto de respuesta
- *status_code* : Después de obtener los datos, podemos comprobar el estado de la operación (éxito, error, etc)
- *encabezados* : Para comprobar los tipos de encabezado
- *Texto* : para extraer el texto del objeto de respuesta recuperado
- *json* : para extraer datos de json Leamos un archivo txt de este sitio web, https://www.w3.org/TR/PNG/iso_8859-1.txt.

```
import requests # importing the request module

url = 'https://www.w3.org/TR/PNG/iso_8859-1.txt' # text from a website

response = requests.get(url) # opening a network and fetching a data
print(response)
print(response.status_code) # status code, success:200
print(response.headers)     # headers information
print(response.text) # gives all the text from the page
<Response [200]>
200
{'date': 'Sun, 08 Dec 2019 18:00:31 GMT', 'last-modified': 'Fri, 07 Nov 2003 05:51:11 GMT', 'etag':
'"17e9-3cb82080711c0;50c0b26855880-gzip"', 'accept-ranges': 'bytes', 'cache-control': 'max-age=31536000',
'expires': 'Mon, 07 Dec 2020 18:00:31 GMT', 'vary': 'Accept-Encoding', 'content-encoding': 'gzip',
'access-control-allow-origin': '*', 'content-length': '1616', 'content-type': 'text/plain', 'strict-transport-security':
'max-age=15552000; includeSubdomains; preload', 'content-security-policy': 'upgrade-insecure-requests'}
```

- Leamos desde una API. La API significa interfaz de programa de aplicación. Es un medio para intercambiar datos de estructura entre servidores principalmente datos de json. Un ejemplo de una API: https://restcountries.eu/rest/v2/all. Leamos esta API utilizando el módulo *de solicitudes*.

```
import requests
url = 'https://restcountries.eu/rest/v2/all'  # countries api
response = requests.get(url)  # opening a network and fetching a data
print(response) # response object
print(response.status_code)  # status code, success:200
countries = response.json()
print(countries[:1])  # we sliced only the first country, remove the slicing to see all countries.
```

```
<Response [200]>
200
[{'alpha2Code': 'AF',
  'alpha3Code': 'AFG',
  'altSpellings': ['AF', 'Afġānistān'],
  'area': 652230.0,
  'borders': ['IRN', 'PAK', 'TKM', 'UZB', 'TJK', 'CHN'],
  'callingCodes': ['93'],
  'capital': 'Kabul',
  'cioc': 'AFG',
  'currencies': [{'code': 'AFN', 'name': 'Afghan afghani', 'symbol': '؋'}],
  'demonym': 'Afghan',
  'flag': 'https://restcountries.eu/data/afg.svg',
  'gini': 27.8,
  'languages': [{'iso639_1': 'ps',
                 'iso639_2': 'pus',
                 'name': 'Pashto',
                 'nativeName': 'پښتو'},
                {'iso639_1': 'uz',
                 'iso639_2': 'uzb',
                 'name': 'Uzbek',
                 'nativeName': 'Oʻzbek'},
                {'iso639_1': 'tk',
                 'iso639_2': 'tuk',
                 'name': 'Turkmen',
                 'nativeName': 'Türkmen'}],
  'latlng': [33.0, 65.0],
  'name': 'Afghanistan',
  'nativeName': 'افغانستان',
  'numericCode': '004',
  'population': 27657145,
  'region': 'Asia',
  'regionalBlocs': [{'acronym': 'SAARC',
                     'name': 'South Asian Association for Regional Cooperation',
                     'otherAcronyms': [],
                     'otherNames': []}],
  'subregion': 'Southern Asia',
  'timezones': ['UTC+04:30'],
  'topLevelDomain': ['.af'],
  'translations': {'br': 'Afeganistão',
                   'de': 'Afghanistan',
                   'es': 'Afganistán',
                   'fa': 'افغانستان',
                   'fr': 'Afghanistan',
                   'hr': 'Afganistan',
                   'it': 'Afghanistan',
                   'ja': 'アフガニスタン',
                   'nl': 'Afghanistan',
                   'pt': 'Afeganistão'}}]
```

Utilizamos el método *json()* del objeto de respuesta, si estamos buscando datos JSON. Para txt, html, xml y otros formatos *de* archivo podemos utilizar *texto*.

10. Crear un paquete.

Organizamos un gran número de archivos en diferentes carpetas y subcarpetas en función de algunos criterios, para que podamos encontrarlos y gestionarlos fácilmente. Como usted sabe, un módulo puede contener múltiples objetos, tales como clases, funciones, etc. Un paquete puede contener uno o más módulos

relevantes. Un paquete es en realidad una carpeta que contiene uno o más archivos de módulo. Vamos a crear un paquete llamado mypackage, siguiendo los siguientes pasos:

Crear una nueva carpeta llamada mypackage dentro de la carpeta 30DaysOfPython Crear un archivo vacío **init.py** en la carpeta mypackage. Crear módulos arithmetic.py y greet.py con el siguiente código:

```
# mypackage/arithmetics.py
# arithmetics.py
def add_numbers(*args):
    total = 0
    for num in args:
        total += num
    return total


def subtract(a, b):
    return (a - b)


def multiple(a, b):
    return a * b


def division(a, b):
    return a / b


def remainder(a, b):
    return a % b


def power(a, b):
    return a ** b
# mypackage/greet.py
# greet.py
def greet_person(firstname, lastname):
    return f'{firstname} {lastname}, welcome to 30DaysOfPython Challenge!'
```

La estructura de carpetas de su paquete debe verse así:

```
─ mypackage
    ├── __init__.py
    ├── arithmetic.py
    └── greet.py
```

Ahora vamos a abrir la shell interactiva de Python y probar el paquete que hemos creado:

```
asabeneh@Asabeneh:~/Desktop/30DaysOfPython$ python
Python 3.9.6 (default, Jun 28 2021, 15:26:21)
[Clang 11.0.0 (clang-1100.0.33.8)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> from mypackage import arithmetics
>>> arithmetics.add_numbers(1, 2, 3, 5)
11
>>> arithmetics.subtract(5, 3)
2
>>> arithmetics.multiple(5, 3)
15
>>> arithmetics.division(5, 3)
1.6666666666666667
```

```
>>> arithmetics.remainder(5, 3)
2
>>> arithmetics.power(5, 3)
125
>>> from mypackage import greet
>>> greet.greet_person('Asabeneh', 'Yetayeh')
'Asabeneh Yetayeh, welcome to 30DaysOfPython Challenge!'
>>>
```

Como podéis ver nuestro paquete funcionan perfectamente. La carpeta del paquete contiene un archivo especial llamado **init.**py - almacena el contenido del paquete. Si ponemos **init.**py en la carpeta del paquete, python start lo reconoce como un paquete. El init**.py** expone los recursos especificados de sus módulos para ser importados a otros archivos de python. Un archivo vacío **init**.py hace que todas las funciones estén disponibles cuando se importa un paquete. El **init.py** es esencial para que la carpeta sea reconocida por Python como un paquete.

Más Información Sobre Los Paquetes

- Base de datos
    - SQLAlchemy o SQLObject - Acceso orientado a objetos a varios sistemas de base de datos diferentes
        - *Pip instalar SQLAlchemy*
- Desarrollo Web
    - Django - Marco web de alto nivel.
        - *Pip instalar django*
    - Flask - micro framework para Python basado en Werkzeug, Jinja 2. (Tiene licencia BSD)
        - *Pip instalar el frasco*
- Analizador HTML
    - Hermosa Sopa - El analizador HTML/XML diseñado para proyectos de respuesta rápida como el raspado de pantalla, aceptará un marcado incorrecto.
        - *Instalación de pip beautifulsoup4*
    - PyQuery - implementa jQuery en Python; más rápido que BeautifulSoup, aparentemente.
- Procesamiento XML
    - ElementTree - El tipo Element es un objeto contenedor simple pero flexible, diseñado para almacenar estructuras de datos jerárquicas, como infosets XML simplificados, en la memoria. --Nota: Python 2.5 y up tiene ElementTree en la biblioteca estándar
- GUI
    - PyQt - Encuadernaciones para el marco Qt multiplataforma.
    - TkInter - El tradicional kit de herramientas de interfaz de usuario de Python.
- Análisis de datos, ciencia de datos y aprendizaje automático
    - Numpy: Numpy (nutrimérico python) es conocida como una de las bibliotecas de aprendizaje automático más populares en Python.
    - Pandas: es un análisis de datos, ciencia de datos y una biblioteca de aprendizaje automático en Python que proporciona estructuras de datos de alto nivel y una amplia variedad de herramientas para el análisis.
    - SciPy: SciPy es una biblioteca de aprendizaje automático para desarrolladores de aplicaciones e ingenieros. La biblioteca SciPy contiene módulos para optimización, álgebra lineal, integración, procesamiento de imágenes y estadísticas.
    - Scikit-Learn: Es NumPy y SciPy. Es considerada como una de las mejores bibliotecas para trabajar con datos complejos.
    - TensorFlow: es una biblioteca de aprendizaje automático creada por Google.
    - Keras: es considerada como una de las bibliotecas de aprendizaje automático más geniales

de Python. Proporciona un mecanismo más fácil para expresar las redes neuronales. Keras también proporciona algunas de las mejores utilidades para compilar modelos, procesar conjuntos de datos, visualizar gráficos y mucho más.

- Red:
    - Solicitudes: es un paquete que podemos utilizar para enviar solicitudes a un servidor (GET, POST, DELETE, PUT)
        - *Solicitudes de instalación de pip*